

notes and decisions explanation

- preprocessing:
 - For simplicity, I selected only the labels that are in the intersection of all: CyNER, DNRTI, and SecureBERT-NER
 - When I preprocessed the data, I noticed that there are multiple lines with two tokens; for example, line_num: 72373 has more than two parts: ['computers.', 'In', 'O']; these were only a few examples, so I fixed them manually (split them and gave each row the correct label.
 - Also, some lines have a label without a token; I ignored them. There were a few cases where there was an I-entity label without a B-entity label before it. For example, ['governmental', 'and', 'diplomatic', 'institutions'] has the label of ['B-Org', 'O', 'I-Org', 'I-Org']. At first, I thought about normalizing it, but when I looked at the example, it made sense, so I decided to leave it like this. If I had more time I would fix the labels to be consistent

- בלייבלים ה gold של dnrti דאטה אני ממפה את כל הלייבלים שאני לא בודקת ל UNK, אבל את הלייבלים שיוצאים מהמודל אם הם לא ברשימת הלייבלים שמעניינים אותי, אני לא ממפה אותם ל UNK אלא משאירה אותם כלייבל O.

בניתי את זה כך כדי שבאוויליואציה בסוף אני אתעלם מכל השורות שה gold label שלהם הוא UNK הם לא ייכנסו לאוויליואציה. לעומת זאת אם המודלים ממפים gold label כלשהו ל O זה נספר כ FN, ולהיפך, אם O ב gold label ממופה ל entity כלשהו שכן ברשימת הלייבלים, זה נספר כ FP. אבל חוץ מזה, פונקציית classification_report איתה אני עושה אוויליואציה, לא מתייחסת ל O כלייבל שמעריכים עליו את איכות המודל

- Comparison:
 - I noticed the models sometimes split one token into multiple tokens for example: Token 'Mandiant' was split into subwords with disagreeing entities [('_Mand', 'B-Malware'), ('iant', 'I-Organization')] In this case, I decided to use the first subword's label (in this case 'B-Malware') for the whole token and merge them into one token. I am not sure if this is a good decision, or if it's better to ignore such cases and not include them in the comparison, but this was my choice given the time. I documented all these cases in a log file saved at logs\model_adapter.log for later debugging
 - In the model_adapter.py file, I created 3 classes: ModelAdapter, SecureBertAdapter (inherits from ModelAdapter), and CyNERAdapter (inherits from ModelAdapter). In this case, it's not very useful to create this structure, and CyNERAdapter and SecureBertAdapter have code repetition. But it can be useful when we want to evaluate another model that has different ways of loading and prediction.

- בפונקציה הזאת אנחנו מודדים את זמן הריצה של predict_labels כלומר גם של ריצת המודל וגם של ה post process. בדיעבד הייתי מכניסה את המדידה לתוך predict_labels ומודדת רק את הזמן של predict אלו הזמנים אחרי השינוי הזה

=== Latency comparison ===

	mean_latency_ms	median_latency_ms	total_time_s	sentences
SecureBERT-NER	93.169	86.476	61.865	664.0
CyNER	119.795	106.979	79.544	664.0

משום מה רק זמני הריצה של secure bert ירדו. אבל לבסוף זאת אותה מסקנה, הפערים בין המודלים אפילו גדלו.

benchmarking report

=== Overall comparison ===

	precision	recall	f1
SecureBERT-NER	0.598	0.603	0.589
CyNER	0.390	0.267	0.292

=== Per-class comparison ===

	SecureBERT-NER				CyNER			
	precision	recall	f1	support	precision	recall	f1	support
INDICATOR	0.816	0.581	0.679	191.0	0.475	0.147	0.224	191.0
MALWARE	0.483	0.457	0.470	317.0	0.196	0.297	0.236	317.0
ORG	0.776	0.735	0.755	818.0	0.704	0.293	0.414	818.0
SYSTEM	0.264	0.488	0.343	209.0	0.010	0.010	0.010	209.0
VULNERABILITY	0.651	0.756	0.700	131.0	0.566	0.588	0.577	131.0

=== Latency comparison ===

	mean_latency_ms	median_latency_ms	total_time_s	sentences
SecureBERT-NER	112.250	96.322	74.309	662.0
CyNER	116.923	105.647	77.403	662.0

From the results, it's clear that SecureBERT is better in all metrics.

If you look at total time latency, there is not much difference, but when you look at the median, the difference is clearer.

Also, the precision and recall are better in total (macro averaging), and even per class. This is a bit suspicious to me; if I had more time, I would test if there are any errors in the mapping.

- השתמשתי ב classification_report אבל רק אחר כך ראיתי שיש ספריה שנקראת nervaluate שנתתם את הלינק אליה.

ניסיתי להריץ את ההשוואה גם עם הפונקציה הזאת ואלו התוצאות.

אפשר לראות ש partial תמיד 0 כי טיפלתי בו בעצמי.

	correct	incorrect	partial	missed	spurious	precision	recall	f1-score
INDICATOR	175	1	0	72	15	0.92	0.71	0.80
MALWARE	198	33	0	84	104	0.59	0.63	0.61
ORG	562	45	0	180	104	0.79	0.71	0.75
SYSTEM	129	29	0	92	204	0.36	0.52	0.42
VULNERABILITY	84	24	0	24	44	0.55	0.64	0.59
SCyNER								
Scenario: strict								
	correct	incorrect	partial	missed	spurious	precision	recall	f1-score
INDICATOR	55	3	0	190	32	0.61	0.22	0.33
MALWARE	128	40	0	147	343	0.25	0.41	0.31
ORG	173	22	0	592	59	0.68	0.22	0.33
SYSTEM	3	8	0	239	150	0.02	0.01	0.01
VULNERABILITY	80	13	0	39	37	0.62	0.61	0.61

call graph

<https://claude.ai/artifact/KCRUh6XgMEHGeK5qGEwzW2>

_configure_file_logging

אם היה לי עוד זמן הייתי מכניסה עוד הדפסות רלוונטיות לתוך ה log למשל ההדפסות ב
read_dnrity_file ב preprocessing.py ובפונקציות נוספות. ה log שהייתי עושה יכול את
שם הפונקציה
שרצה ואת זמן האירוע.